

Strutture dati non lineari

Martedì 10 Marzo 2026



Una struttura dati si definisce **dinamica** se permette di rappresentare insiemi dinamici la cui cardinalità varia durante l'esecuzione del programma

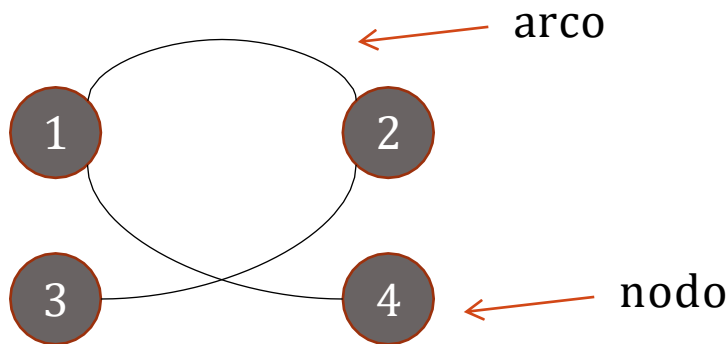
Strutture dinamiche non lineari:

- Grafi
- alberi
- alberi binari



GRAFO

- Un grafo è una coppia $G=(V,E)$
 - V è l'insieme dei nodi o vertici
 - E è un insieme di coppie di vertici. Tali coppie sono dette archi
- Per esempio:
 - $V=\{1,2,3,4\}$
 - $E=\{ \{1,2\} , \{2,3\} , \{1,4\} \}$
- Rappresentazione :





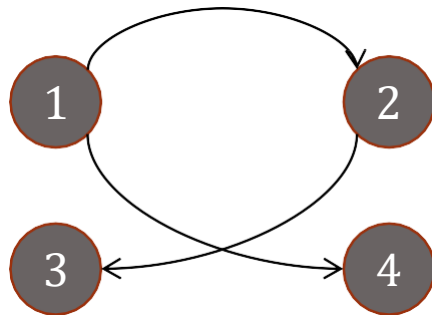
GRAFO NON ORIENTATO

- Un grafo non orientato è un grafo in cui gli archi sono coppie non ordinate di vertici, cioè un arco fra i vertici u, v è un insieme di due elementi $\{u, v\}$ piuttosto che una coppia (u, v)
- Tuttavia spesso si indica l'arco con notazione (u, v) precisando che si tratta di un grafo non orientato



GRAFO ORIENTATO

- Se le coppie di E sono ordinate, ovvero $E \subseteq V \times V$, il grafo si dice orientato o diretto (in caso contrario si dice non orientato)
- Per esempio:
 - $V = \{1, 2, 3, 4\}$
 - $E = \{ (1, 2), (2, 3), (1, 4) \}$
- Rappresentazione :



ARCO

- Sia (u,v) è un arco di un grafo orientato, si dice che:
 - l'arco esce dal vertice u
 - l'arco entra nel vertice v
- Un arco (u,v) di un grafo non orientato si dice che è incidente sui vertici v e u e si dice che v è adiacente a u .
 - in un grafo non orientato la relazione di adiacenza è simmetrica
 - in un grafo orientato se esiste un arco da u a v allora v è adiacente a u , ma non è detto che sia vero il viceversa



CAMMINI E CICLI

- Due vertici u, v connessi da un arco e prendono nome di **estremi dell'arco**.
- Il grado di un nodo x è il numero di archi che hanno x come estremo. Se il grafo è orientato è il numero di archi uscenti da x .
- Un arco che ha due estremi coincidenti si dice **cappio**.
- Un **cammino** è dato da una successione di vertici $v_0, v_1, \dots, v_n$ e da una successione di archi che li collegano $(v_0, v_1), (v_1, v_2), \dots, (v_{n-1}, v_n)$. I vertici v_0 e v_n si dicono *estremi* del percorso. La lunghezza del cammino è il numero degli archi che lo compongono.
- Un cammino è semplice se tutti i vertici sono distinti.
- In un grafo orientato un cammino è un ciclo se $v_0 = v_n$.
- Un grafo si dice aciclico se non contiene cicli



CONNETTIVITÀ DI UN GRAFO

- Due nodi u e v di un grafo si dicono connessi se esiste un cammino che li congiunge.
- Un grafo non orientato è connesso se comunque presi due nodi, essi sono connessi
- Un grafo orientato si dice fortemente connesso se comunque presi due nodi u e v esiste una cammino da u a v e da v a u .



Come implementiamo un Grafo?



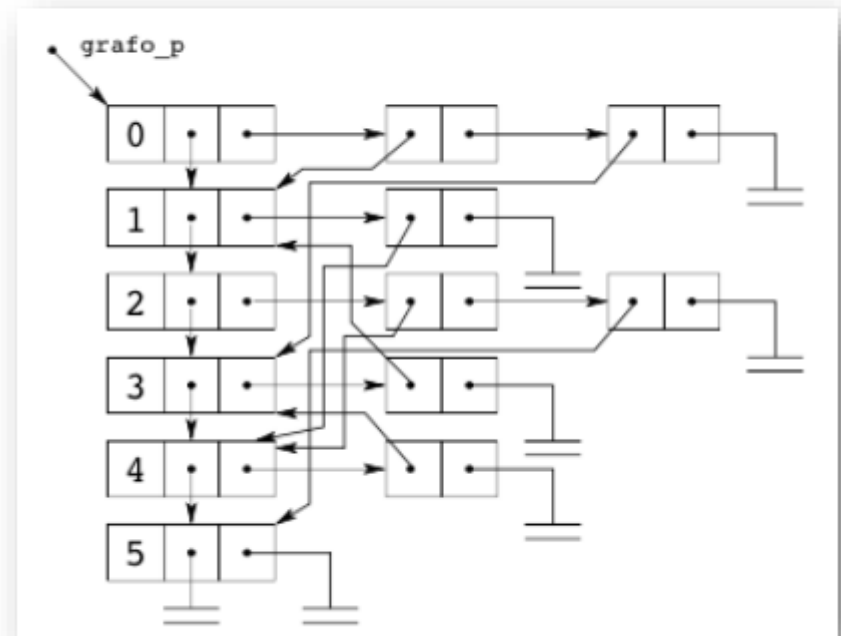
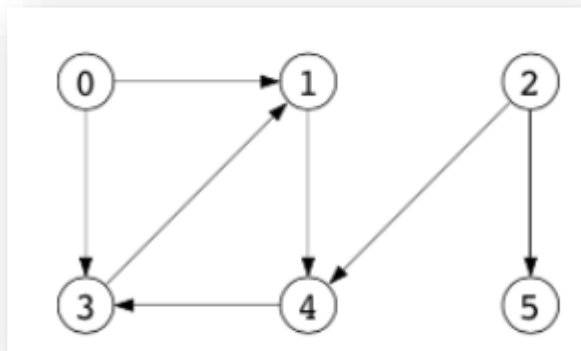
Implementazione con Lista di Adiacenza

il grafo viene rappresentato come una struttura dati dinamica reticolare detta **lista di adiacenza**, formata da una lista primaria dei vertici e più liste secondarie degli archi

- la lista primaria contiene un elemento per ciascun vertice del grafo, il quale contiene a sua volta la testa della relativa lista secondaria
- la lista secondaria associata ad un vertice descrive tutti gli archi uscenti da quel vertice



Implementazione con Lista di Adiacenza

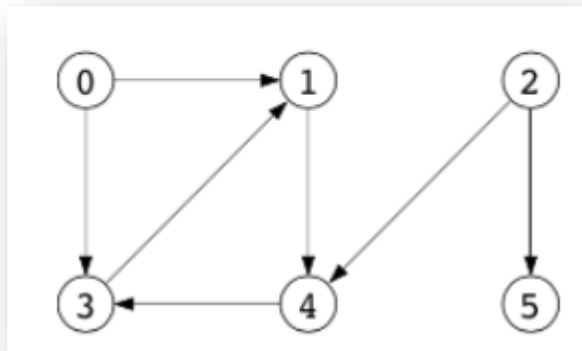


Implementazione con Matrice di Adiacenza

- se la struttura di un grafo non cambia oppure è importante fare accesso rapidamente alle informazioni contenute nel grafo, allora conviene ricorrere ad una rappresentazione a matrice di adiacenza.
- la matrice ha tante righe e tante colonne quanti sono i vertici o l'elemento di indici i e j vale 1 se esiste un arco dal vertice i al vertice j , 0 altrimenti
- per i grafi pesati si può sostituire il valore 1 con il peso del grafo



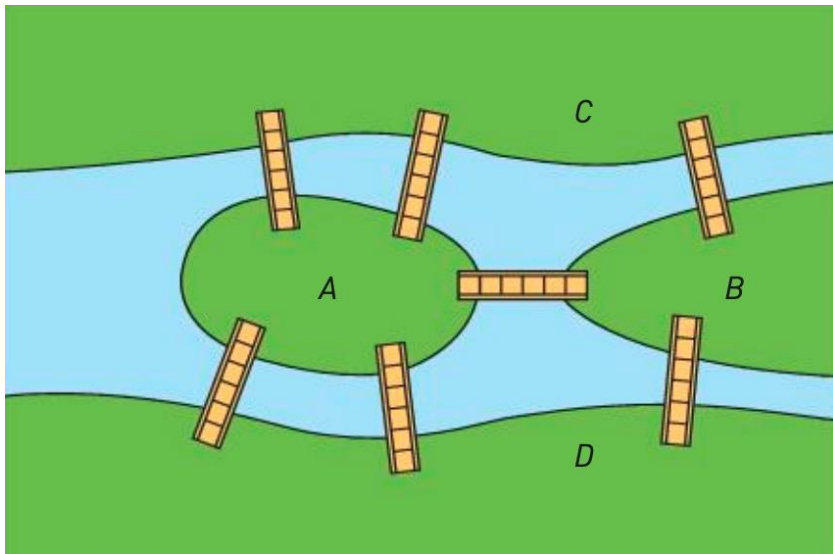
Implementazione con Matrice di Adiacenza



0	1	0	1	0	0
0	0	0	0	1	0
0	0	0	0	1	1
0	1	0	0	0	0
0	0	0	1	0	0
0	0	0	0	0	0



Applicazione dei grafi: esempio



il problema dei sette ponti di Königsberg è un problema ispirato da una città reale e da una situazione concreta o Königsberg è percorsa dal fiume Pregel e da suoi affluenti e presenta due estese isole che sono connesse tra di loro e con le due aree principali della città da sette ponti o nel corso dei secoli è stata più volte proposta la questione se sia possibile con una passeggiata seguire un percorso che attraversi ogni ponte una e una volta soltanto e tornare al punto di partenza o nel 1736 Leonhard Euler affrontò tale problema, dimostrando che la passeggiata ipotizzata non era possibile

<https://www.ilpost.it/2023/03/02/sette-ponti-teoria-grafi/>



Astrazione

Eulero ha formulato il problema in termini di teoria dei grafi, astraendo dalla situazione specifica di Königsberg o eliminazione di tutti gli aspetti contingenti ad esclusione delle aree urbane delimitate dai bracci fluviali e dai ponti che le collegano o rappresentazione di ogni area urbana con un vertice o rappresentazione di ogni ponte con un arco



ALBERO: DEFINIZIONE

- Un albero è un grafo non orientato connesso e aciclico
- Un albero soddisfa le seguenti proprietà:
 - $|E| = |V| - 1$;
 - se viene aggiunto un nuovo arco si forma un ciclo;
 - se viene tolto un arco si perde la connessione
 - comunque presi due nodi esiste un unico cammino che li congiunge



ALBERI RADICATI

- Un *albero* (in inglese *tree*) è una struttura dati largamente utilizzata in computer science, che astrae il concetto di gerarchia
- Un albero è una generalizzazione di una lista concatenata in cui i nodi stanno in una relazione di tipo padre-figlio
- Generalmente si disegnano elencando la gerarchia dei nodi dall'alto verso il basso (pensate a un albero genealogico)
- Un classico esempio di albero in informatica è l'albero delle directory



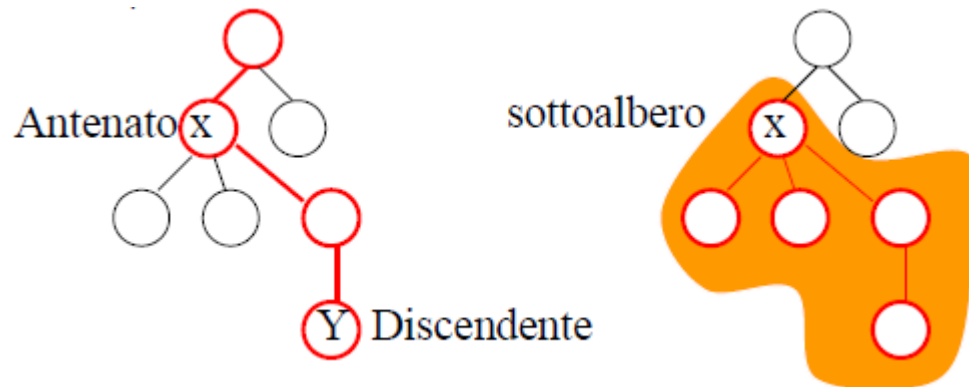
ALBERO RADICATO

- Un albero radicato è un albero in cui un nodo è designato come radice (root)
- I nodi sono collegati da *archi* (*edges*)
- In un albero radicato il livello di un nodo è la distanza del nodo dalla radice, ovvero la lunghezza dell'unico cammino che congiunge la radice al nodo



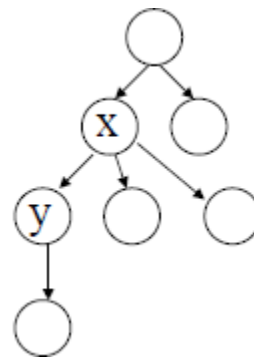
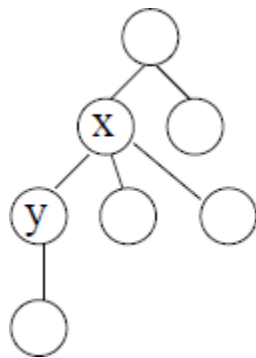
ANTENATI E DISCENDENTI

- Sia x un nodo di un albero con radice r . Qualunque nodo x sull'unico cammino da r a y è chiamato antenato (ancestor) di y e y si dice discendente (descendant) di x
- Il sottoalbero radicato in x è l'albero indotto dai discendenti di x , radicato in x (ovvero l'albero formato da tutti i nodi discendenti di x , x stesso e relativi archi) si trovano nel livello sotto).



RADICE E ORIENTAMENTO

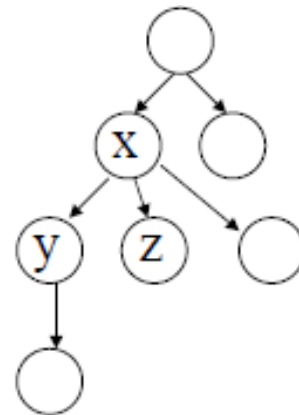
- Una radice induce un orientamento
- Si può cioè predicare l'attributo di predecessore e successore su ogni nodo
- Si può rappresentare l'orientamento con frecce



RELAZIONE FRA I NODI

- Se l'ultimo arco di un cammino dalla radice r ad un nodo x è l'arco (x,y) allora x è il padre (parent) di y e y è il figlio (child) di x
- Il numero di figli di un nodo x è il grado di x
- Due nodi con lo stesso padre si dicono fratelli (simbling)

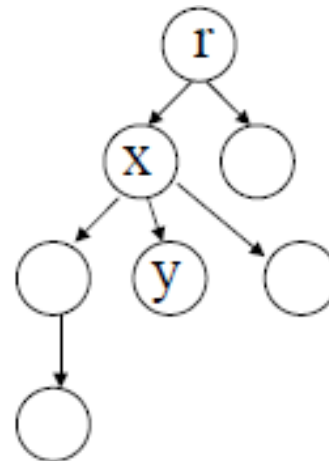
x =padre di y
 y =figlio di x
 y fratello di z



NODI PARTICOLARI

- La radice è l'unico nodo che non ha padre
- I nodi senza figli si dicono nodi esterni o foglie (leaves)
- Un nodo non foglia è un nodo interno (internal node)
- La dimensione di un albero è il numero dei suoi nodi

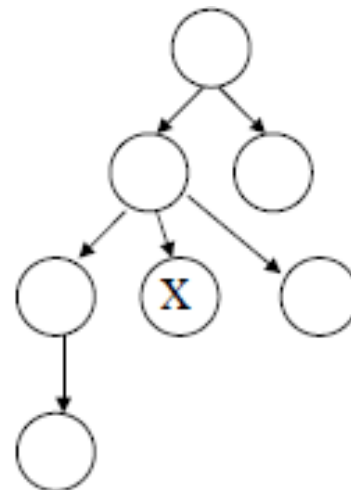
r=radice
x=nodo interno
y=foglia



ALTEZZA

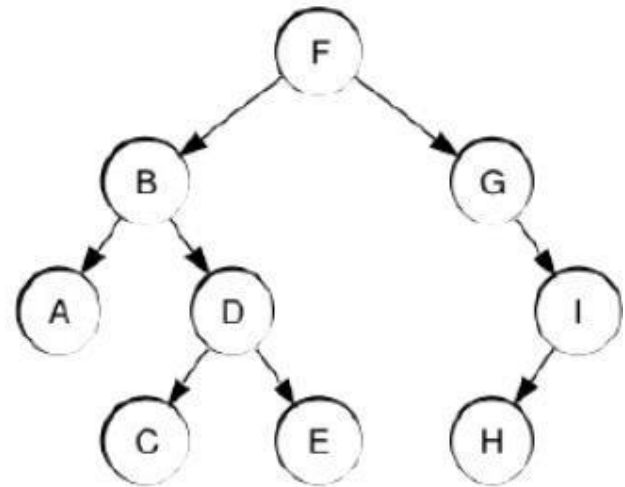
- La lunghezza di un cammino da r a x è la profondità di x
- La profondità massima di un qualunque nodo di un albero è l'altezza (height) dell'albero

Profondità $x=2$
Altezza=3



ESEMPIO

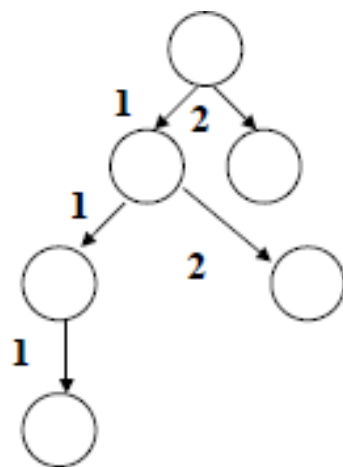
- Il nodo F è la radice, G un nodo interno, A una foglia
- E e C sono fratelli, il loro padre è D
- Esiste un cammino da B a C, ma non da A ad E
- Infatti B è un antenato di C, ma E non è un discendente di A
- La profondità di D è 2, quella di F è 0
- L'altezza di B è 2, quella di F è 3
- L'albero ha altezza 3 e dimensione 9



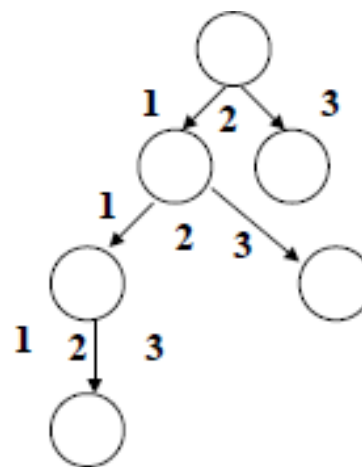
ALBERI ORDINATI E POSIZIONALI

- Un albero ordinato è un albero radicato in cui i figli di ciascun nodo sono ordinati (cioè si distingue il primo figlio, il secondo, etc)
- Un albero si dice posizionale se i figli dei nodi sono etichettati con interi positivi distinti (l'i-esimo figlio di un nodo è assente se nessun figlio è etichettato con l'intero i)

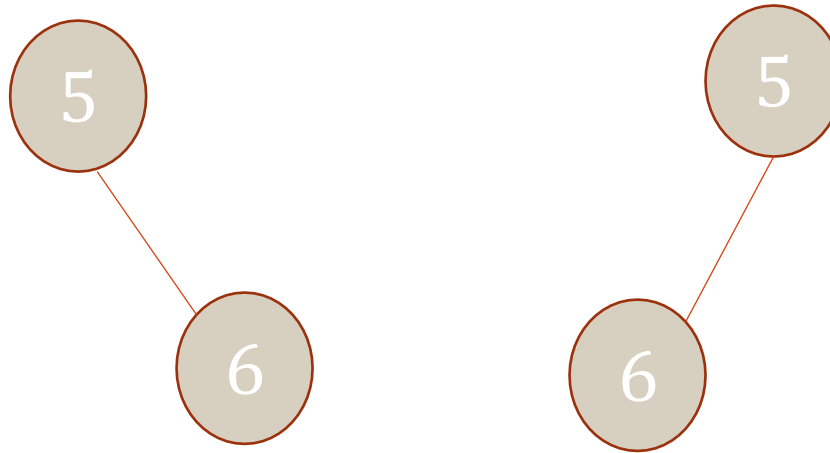
ordinato



posizionale



PER ESEMPIO



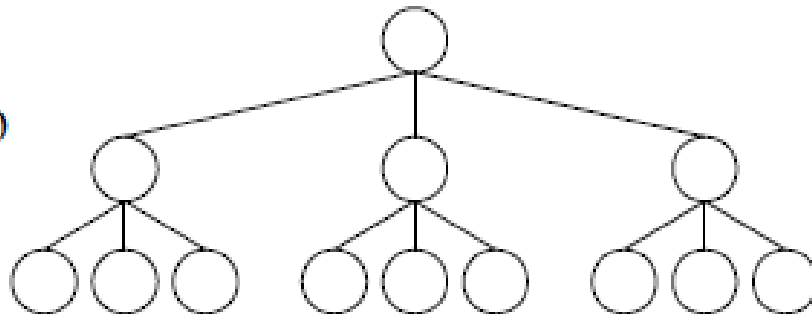
- In un albero ordinato non si distingue fra figlio destro o sinistro (ma si considera solo il numero di figli e le loro etichette) ...mentre in albero posizionale sì
- I due alberi, coincidono come alberi ordinati ma sono diversi come alberi posizionali.



ALBERO K-ARIO

- Un albero k-ario è un albero posizionale in cui per ogni nodo tutti i figli con posizione più grande di k sono assenti (tutti i nodi interni hanno al più k figli)
- Un albero k-ario completo è un albero k-ario in cui tutte le foglie hanno la stessa profondità e tutti i nodi interni hanno grado k

Albero 3-ario



PROPRIETÀ DI UN ALBERO K-ARIO COMPLETO

- Il numero di foglie di un albero k-ario completo di altezza h è k^h (**si prova per induzione su h**).
Osserviamo che:
 - la radice ha k figli a profondità 1
 - ognuno dei figli ha k figli a profondità 2 per un totale di $k*k$ figli
 - a profondità h si hanno k^h foglie
- Il numero di nodi interni di un albero k-ario completo di altezza h è:
 - $1+k+ k^2 + \dots +k^{h-1} = (k^h -1)/(k-1)$



ALBERO BINARIO

- Un albero binario è un albero k-ario con $k=2$
- Quindi:
 - ogni nodo ha al più due figli
 - un albero binario completo è un albero in cui ogni nodo interno ha esattamente due figli e le foglie hanno tutte la stessa profondità.



DEFINIZIONE RICORSIVA DI ALBERO BINARIO

- Un albero binario è :
 - una struttura vuota, oppure
 - una struttura composta da tre insiemi disgiunti di nodi: un nodo radice, un albero binario chiamato **sottoalbero sinistro** e un albero binario chiamato **sottoalbero destro**



OSSERVAZIONI SUGLI ALBERI BINARI

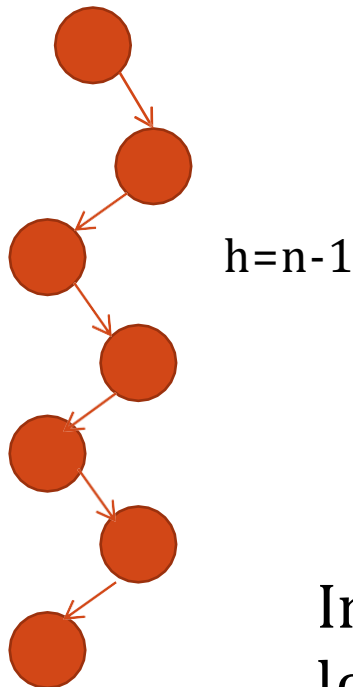
- Un albero binario
 - non è un albero ordinato con nodi di grado al più due
 - ma è un albero posizionale con nodi di grado al più due
- Un albero binario completo di altezza h ha $2^h - 1$ nodi interni
- Un albero binario completo di altezza h ha 2^h foglie
- Il numero totale di nodi in un albero binario completo di altezza h è $2^{h+1} - 1$



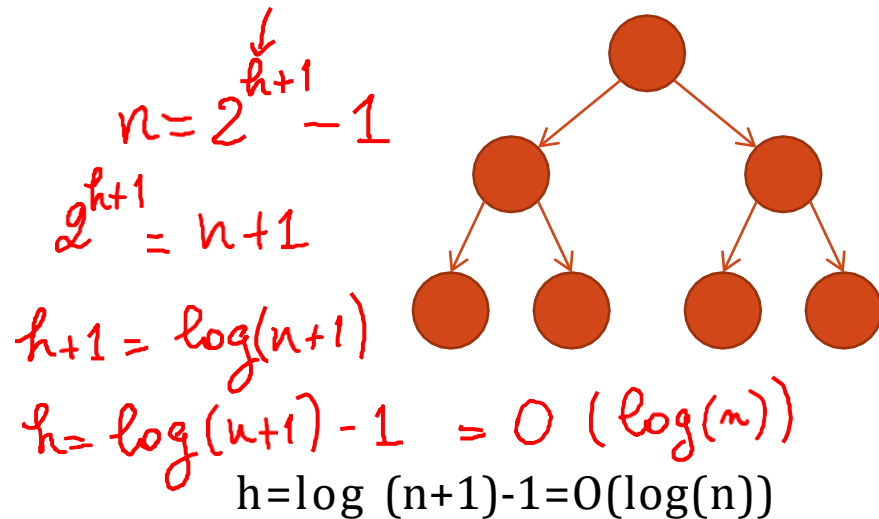
ALTEZZA DI UN ALBERO CON N NODI

- Come distribuire n nodi su un albero?
Quale altezza si ottiene?

ALBERO LINEARE



ALBERO COMPLETO



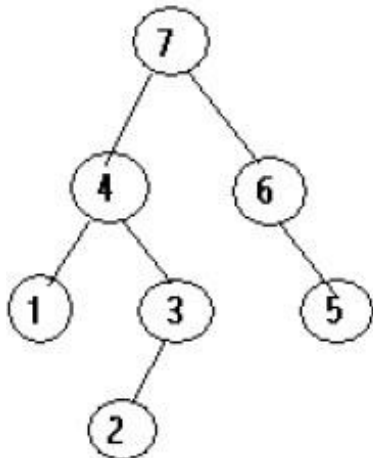
In generale,
 $\log(n + 1) - 1 \leq h \leq n - 1$



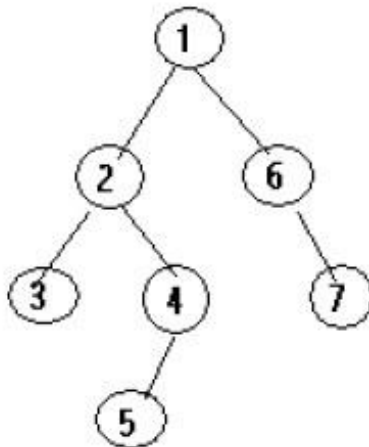
VISITE DI UN ALBERO

- Per visitare i nodi di un albero esistono diversi ordini:
 - visita in post-ordine
 - visita in pre-ordine
 - visita simmetrica (solo per alberi binari)

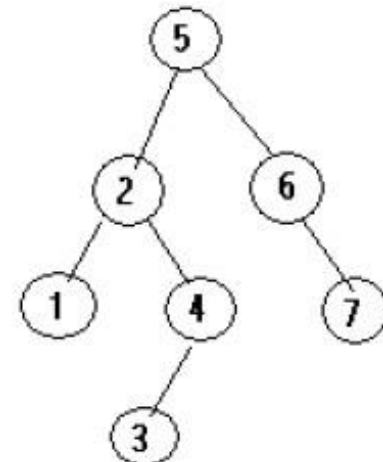
Postorder



Preorder

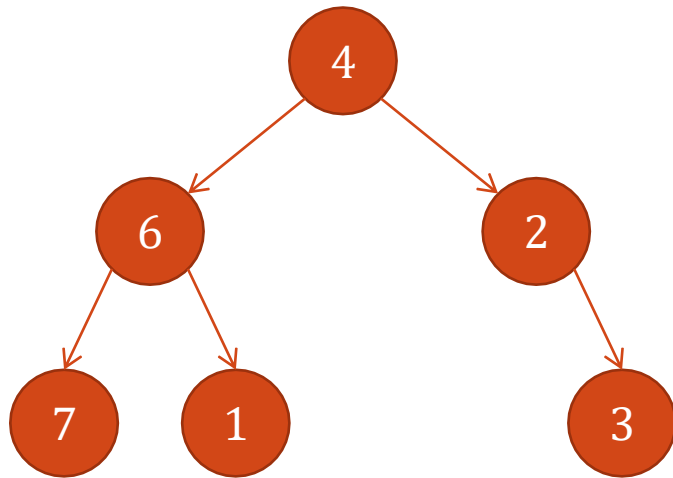


Inorder



VISITA IN PREORDINE DI UN ALBERO BINARIO

- Si visita la radice, poi si visita in preordine il sottoalbero sinistro e poi il sottoalbero destro



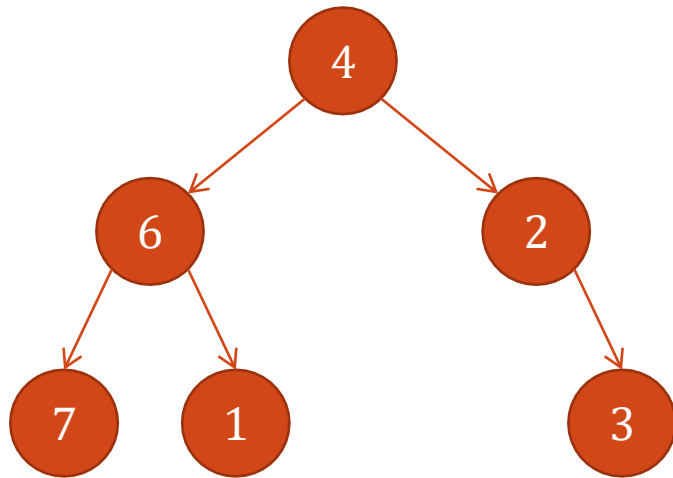
```
Algoritmo pre-order (nodo v)
{ visita (v);
  if v ha figlio sinistro vleft
    pre-order (vleft)
  if v ha figlio destro vright
    pre-order (vright)
}
```

Visita in preordine: 4 6 7 1 2 3



VISITA IN ORDINE SIMMETRICO DI UN ALBERO BINARIO

- Si visita in ordine simmetrico il sottoalbero sinistro, poi la radice e poi, in ordine simmetrico, il sottoalbero destro



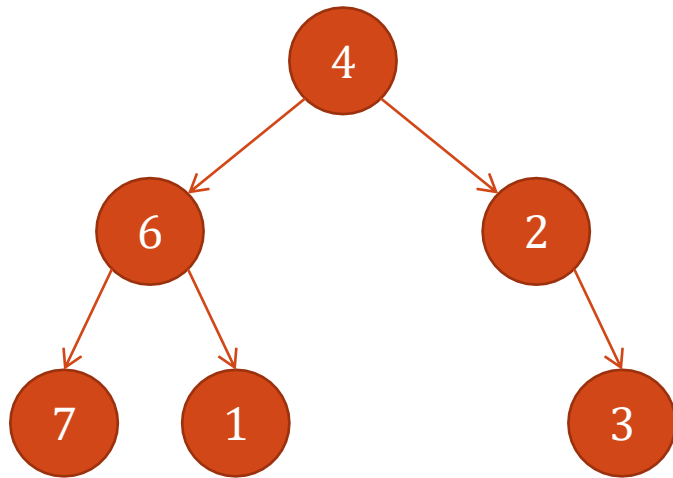
```
Algoritmo in-order (nodo v)
{ if v ha figlio sinistro vleft
  in-order (vleft)
  visita (v);
  if v ha figlio destro vright
    in-order (vright)
}
```

Visita in ordine simmetrico: 7 6 1 4 2 3



VISITA IN POSTORDINE DI UN ALBERO BINARIO

- Si visita in postordine il sottoalbero sinistro, poi il sottoalbero destro e poi la radice



```
Algoritmo post-order (nodo v)
{ if v ha figlio sinistro vleft
  post-order (vleft)
  if v ha figlio destro vright
    post-order (vright)
  visita (v);
}
```

Visita in postordine: 7 1 6 3 2 4



ALCUNI PROBLEMI SUGLI ALBERI

- Descrivere un algoritmo per la visita per livelli di un albero binario
- Descrivere un algoritmo che date le visite in preordine e in ordine simmetrico, costruisca l'albero binario corrispondente.



```

class AlberoBin {
public:
    AlberoBin();
    AlberoBin(string i);
    AlberoBin(string i, AlberoBin* l,
                AlberoBin* r);
    virtual ~AlberoBin();
    ... <setter & getter>
    void preOrder();
    void inOrder();
    void postOrder();
private:
    string info;
    AlberoBin* left;
    AlberoBin* right;
};

```

```

void AlberoBin::preOrder() {
    cout << info << " - ";
    if (left!=nullptr)    left->preOrder();
    if (right!=nullptr)   right->preOrder();
}

void AlberoBin::inOrder() {
    if (left!=nullptr)    left->inOrder();
    cout << info << " - ";
    if (right!=nullptr)   right->inOrder();
}

void AlberoBin::postOrder() {
    if (left!=nullptr)    left->postOrder();
    if (right!=nullptr)   right->postOrder();
    cout << info << " - ";
}

```

